

[NJC/2025/Q1]

1 Name your Jupyter Notebook as `Task1_<your name>.ipynb`.

The task is to use Object-Oriented Programming (OOP) to implement two data structures that store their data using a fixed-size array:

- A queue data structure follows the First-In-First-Out (FIFO) principle, meaning that elements are removed in the same order in which they were added.
- A stack data structure follows the Last-In-First-Out (LIFO) principle, meaning that the most recently added element is the first one to be removed.

For each sub-task, add a comment statement, at the beginning of the code using the hash symbol "#", to indicate the sub-task the program code belongs to, for example:

```
In [1]: 

#Task 1.1



Program code

  
Output:
```

Task 1.1

The class `Queue` contains the following attributes:

- `size`: the maximum number of items that can be inserted into the queue.
- `data`: a one-dimensional array of objects.
- `front`: the index in the `data` array that points to the first item in the queue.
- `back`: the index in the `data` array that points to the last item in the queue.
- `in_use`: the number of items currently in the queue.

[NJC/2025/Q1 Cont ']

The class `Queue` has the following methods:

- `constructor`: takes an integer, `n`, as a parameter and initialises the array data with `n` number of `None` values. It also initialises the attributes `front`, `back`, and `in_use` to appropriate values for an empty queue
- `is_empty()`: returns `True` if the queue is empty, `False` otherwise.
- `is_full()`: returns `True` if the queue is full, `False` otherwise.
- `enqueue(item)`: takes an `item` as a parameter and inserts it into the queue. If the queue is full, prints an error message “Queue full”.
- `dequeue()`: removes the first item in the queue and returns the removed item. If the queue is empty, prints an error message “Queue empty” and return `None`.
- `__repr__()`, returns a string representation of the items in the queue in the format:

[<item_1>, <item_2>, ... <item_n>], where <item_1> is the data item at the front of the queue and <item_n> is at the back of the queue. Only items in the queue should be displayed.

Write Python code to implement the class `Queue` **as a circular queue**, based on the attributes and methods described above.

[15]

[NJC/2025/Q1 Cont ']

Task 1.2

Write Python code to test the functionality of the class `Queue` as follows:

- create an object instance of the class `Queue` with a size of **three**.
- insert **four** items into the queue using the `enqueue` method and print the contents of the queue.
- remove **two** items from the queue using the `dequeue` method and print the items that was removed.
- insert **one** more item to the queue and then print the contents of the queue. [1]

Task 1.3

In this task, you are to implement a **stack** data structure using **two queue objects**.

This means that the two stack operations, push and pop, must be implemented **using only the operations provided by queues**. The stack should behave exactly as a normal stack would (LIFO), but internally, items must be stored and accessed using the two queues.

The class `Stack` is to be implemented with the following attributes:

- `size`: the maximum number of items that can be inserted into the stack.
- `queue_1, queue_2`: instances of the `Queue` class as implemented in Task 1.1
- `in_use`: the number of items currently in the stack.

The class `Stack` has the following methods:

- `constructor`: initialises the attributes of the class `Stack`.
- `is_empty()`: returns `True` if the stack is empty, `False` otherwise.
- `is_full()`: returns `True` if the stack is full, `False` otherwise.
- `push(item)`: takes an `item` as a parameter and inserts it into the stack. If the stack is full, prints an error message "Stack full".
- `pop()`: removes the item at the top of the stack and returns it. If the stack is empty, prints an error message "Stack empty" and return `None`.

[NJC/2025/Q1 Cont ']

- `__repr__()`: returns a string representation of the items in the stack in the format:

`[<item_1>, <item_2>, ... <item_n>]`, where `<item_1>` is at the top of the stack and `<item_n>` is at the bottom. Only items in the stack should be displayed.

Write Python code to implement the `Stack` class. In this task,

- you are NOT ALLOWED to add any new attributes.
- No additional storage space is allowed (e.g. creating a new list) to solve the problem.
- You are NOT ALLOWED to access the attributes of the `Queue` class directly. Only the methods provided by the `Queue` class are allowed to be used.

[12]

Task 1.4

Write Python code to test the functionality of the class `Stack` as follows:

- create an object instance of the class `Stack` with a size of **three**.
- insert **four** items into the stack using the `push()` method and print the contents of the stack.
- remove **one** item from the stack using the `pop()` method and print the item that was removed.
- insert **one** more item into the stack and then print the contents of the stack. [1]

Save your Jupyter Notebook for Task 1.